
继承

复习

1. 修饰符

重点

2. 友元类与友元函数

重点

3. 运算符重载

重点

目录

- | | |
|-------------------|----|
| 1. 继承的基本概念和派生类的定义 | 了解 |
| 2. 继承的语法规则 | 熟悉 |
| 3. 三种继承方式 | 重点 |
| 4. 继承中的构造和析构 | 重点 |
| 5. 多继承 | 重点 |
| 6. 多继承构造函数 | 重点 |
| 7. 多继承命名冲突 | 重点 |
| 8. 菱形继承 | 重点 |
| 9. 虚继承 | 重点 |
| 10. 虚继承构造函数 | 重点 |

继承的基本概念和派生类的定义

当你创建了一个动物类，动物类有吃喝睡跑等动作；这个时候你的老板又说需要创建一个狗的类，同样要有吃喝睡跑等动作。

你是选择重新写一个狗的类？

还是选择把动物类的代码copy来改改？

继承的基本概念和派生类的定义

继承使用场景

如果创建的新类与现有的类相似，只是多出若干成员变量或成员函数时，可以使用**继承**，这样不但会减少代码量，而且新类会拥有基类的所有功能。

如果需要创建多个类，它们拥有很多相似的成员变量或成员函数时，也可以使用继承。

可以将这些类的共同成员提取出来，定义为基类，然后从基类继承，既可以节省代码，也方便后续修改成员。

继承的基本概念和派生类的定义

C++ 中的继承是类与类之间的关系，是一个很简单很直观的概念，与现实世界中的继承类似，比如儿子继承父亲的财产。

继承（Inheritance）可以理解为一个类从另一个类获取成员变量和成员函数的过程。例如类 B 继承于类 A，那么 B 就拥有 A 的成员变量和成员函数。

在C++中，**派生（Derive）和继承是一个概念**，只是站的角度不同。继承是儿子接收父亲的产业，派生是父亲把产业传承给儿子。

被继承的类称为父类或基类，继承的类称为子类或派生类。**“子类”和“父类”通常放在一起称呼，“基类”和“派生类”通常放在一起称呼。**

继承的语法规则

语法

class derived-class: access-specifier base-class

derived-class 派生类，即继承自 base-class 创建的新类。

access-specifier 继承访问修饰符。

base-class 基类，即要被继承的类。

继承案例

创建了一个新类 Dog，其继承自 Animal。

继承案例 代码

```
class Animal {
public:
    string name;
    int age;

public:
    Animal() {}
    Animal(string name, int age) {
        this->name = name;
        this->age = age;
    }
    void printInfo() {
        cout << "名字叫" << name << " , 岁数是" << age << endl;
    }
};
```

```
class Dog :public Animal {
public:
    Dog() {}
    Dog(string name, int age, char sex) {
        this->name = name;
        this->age = age;
        this->sex = sex;
    }

public:
    char sex;
};

int main() {

    Dog d1("xg",2,'m');
    d1.printInfo();
    return 0;
}
```

三种继承方式

C++ 中的 继承 有三种方式，即 `public`、`protected` 和 `private`，此项是可选项，如果不写，默认为 `private`，也就是成员变量和成员函数默认也是 `private`。

三种继承方式

不同的继承方式会影响基类成员在派生类中的访问权限。

public继承方式

1. 基类中所有 public 成员在派生类中为 public 属性；
2. 基类中所有 protected 成员在派生类中为 protected 属性；
3. 基类中所有 private 成员在派生类中不能使用。

protected继承方式

1. 基类中的所有 public 成员在派生类中为 protected 属性；
2. 基类中的所有 protected 成员在派生类中为 protected 属性；
3. 基类中的所有 private 成员在派生类中不能使用。

private继承方式

1. 基类中的所有 public 成员在派生类中均为 private 属性；
2. 基类中的所有 protected 成员在派生类中均为 private 属性；
3. 基类中的所有 private 成员在派生类中不能使用。

三种继承方式

先创建一个Person类，把name，age，height声明为不同的权限。

三种继承方式

先创建一个Person类，把name，age，height声明为不同的权限。

```
class Person {  
public:  
    string name;  
protected:  
    int age;  
private:  
    double height;  
public:  
    void setName(string name) {  
        this->name = name;  
    }  
    void setAge(int age) {  
        this->age = age;  
    }  
    void setHeight(int height) {  
        this->height = height;  
    }  
    void printInfo() {  
        cout << "name = " << this->name << " age = " << this->age << " height = " << this->height << endl;  
    }  
};
```

三种继承方式

定义了一个派生类 Student，其继承自基类 Person，同时，我们使用了 public 继承。



代码有什么问题吗？

三种继承方式

定义了一个派生类 Student，其继承自基类 Person，同时，我们使用了 public 继承。

```
class Student :public Person {  
  
};  
int main() {  
    Student s1;  
    s1.setName("zs");  
    s1.setAge(20);  
    s1.setHeight(190);  
    s1.printInfo();  
  
    return 0;  
}
```



代码有什么问题
吗？

三种继承方式

直接调用了 Person 类里面的 public 方法，此时程序运行正常。

三种继承方式

直接调用了 Person 类里面的 public 方法，此时程序运行正常。

现在修改程序，在派生类中直接访问基类的成员，

三种继承方式

现在修改程序，在派生类中直接访问基类的成员，

```
class Student : public Person
{
public:
    void showInfo()
    {
        cout << "Stu Name = " << this->name << " Age = " << this->age << " Height = " << this->height << endl;
    }
};

int main() {
    Student s1;
    s1.setName("zs");
    s1.setAge(20);
    s1.setHeight(190);
    s1.printInfo();
    s1.showInfo();

    return 0;
}
```

private成员不可访问

三种继承方式

再次修改程序，修改为 private 继承

```
class Student : private Person
{
};
int main() {
    Student s1;
    s1.setName("zs");
    s1.setAge(20);
    s1.setHeight(190);
    s1.printInfo();
    s1.showInfo();

    return 0;
}
```

修改为了 private
现在可以看到不可以访问基类里面的任何成员了。

三种继承方式

继承方式中的 public、protected、private 是用来指明基类成员在派生类中的最高访问权限的。不管继承方式如何，基类中的 private 成员在派生类中始终不能使用（不能在派生类的成员函数中访问或调用）。

如果希望基类的成员能够被派生类继承并且毫无障碍地使用，那么这些成员只能声明为 public 或 protected；只有那些不希望在派生类中使用的成员才声明为 private。如果希望基类的成员既不向外暴露（不能通过对象访问），还能在派生类中使用，那么只能声明为 protected。

继承中的构造和析构

C++ 中，在使用继承时，子类可以继承所有父类的非 private 的成员函数和成员变量，**但子类不能继承父类的构造函数。**

为什么不能继承？即使继承了，它的名字和派生类的名字也不一样，不能成为派生类的构造函数，当然更不能成为普通的成员函数。

虽然，构造函数不会被继承过来，但是在做子类初始化时，很多场景都需要使用到父类的构造函数，此时，还是可以在子类中调用父类的构造函数的。

继承中的构造和析构

子类调用父类构造函数

语法

```
ChildClass::ChildClass(string name, int age, float score): SuperClass(name, age), m_score(score)
{
}
```

在子类 ChildClass 的构造函数中，调用了父类 SuperClass 的构造函数。

继承中的构造案例

```
class Person
{
public:
    Person(string name, int age)
    {
        this->name = name;
        this->age = age;
    }
    void info()
    {
        cout << "person name = " << this->name << " age = " << this->age << endl;
    }
protected:
    string name;
    int age;
};
class Student : public Person
{
public:
    Student(string name, int age, float score) : Person(name, age), score(score)
    {
    }
    void info()
    {
        cout << "student name = " << this->name << " age = " << this->age << " score = " << this->score << endl;
    }
private:
    float score;
};
int main() {
    Student stu("zs", 21, 99.5);
    stu.info();
    return 0;
}
```

通过using来继承基类构造函数

C++11标准中，可通过using Base::Base把基类构造函数继承到派生类中，不再需要书写多个派生类构造函数来完成基类的初始化。更为巧妙的是，C++11标准规定，继承构造函数与类的一些默认函数（默认构造、析构、拷贝构造函数等）一样，是隐式声明，如果一个继承构造函数不被相关代码使用，编译器不会为其产生真正的函数代码。

通过using来继承基类构造函数

```
class Person
{
public:
    Person(string name, int age)
    {
        this->name = name;
        this->age = age;
    }
    void info()
    {
        cout << "person name = " << this->name << " age = " << this->age << endl;
    }
protected:
    string name;
    int age;
};
class Student : public Person
{
public:
    using Person::Person;
    Student(string name, int age, float score) : Person(name, age), score(score)
    {
    }
    void info()
    {
        cout << "student name = " << this->name << " age = " << this->age << " score = " << this->score << endl;
    }
private:
    float score;
};
int main() {
    Student stu("zs", 21, 99.5);
    stu.info();
    return 0;
}
```

构造函数调用顺序

子类 继承 父类时，当实例化子类时，一定会优先调用父类的 构造函数，事实上，通过派生类创建对象时必须调用基类的构造函数，这是语法规定。

换句话说，定义派生类构造函数时最好指明基类构造函数；如果不指明，就调用基类的默认构造函数（不带参数的构造函数）；**如果没有默认构造函数，那么编译失败。**

构造函数调用顺序

```
class Animal {
public:
    Animal() {
        cout << "call Aniaml" << endl;
    }
};
class Monkey :public Animal {
public:
    Monkey() {
        cout << "call Monkey" << endl;
    }
};

int main() {
    Monkey m1;
    return 0;
}
```

```
call Aniaml
call Monkey
```

构造函数调用顺序

```
class Animal {
public:
    Animal() {
        cout << "call Aniaml" << endl;
    }
};
class Monkey :public Animal {
public:
    Monkey() {
        cout << "call Monkey" << endl;
    }
};
class GoldenMonkey :public Monkey {
public:
    GoldenMonkey() {
        cout << "call GoldenMonkey" << endl;
    }
};

int main() {
    GoldenMonkey gm1;
    return 0;
}
```

```
call Aniaml
call Monkey
call GoldenMonkey
```

构造函数调用顺序

子类继承父类时，实例化子类，如果不知道基类构造函数，则使用默认构造函数，如果没有，则报错

构造函数调用顺序

```
class Animal {
public:
    Animal(string name) {
        cout << "call Aniaml" << endl;
    }
};

class Monkey :public Animal {
public:
    Monkey() {
        cout << "call Monkey" << endl;
    }
};

int main() {
    Monkey m1;
    return 0;
}
```

	代码	说明
	E0291	类 "Animal" 不存在默认构造函数
	C2512	"Animal" : 没有合适的默认构造函数可用

继承中的构造和析构

在 C++ 中，使用子类 继承 父类时，当析构子类时，一定会优先调用子类的 析构函数，接着，才会调用父类的析构函数，正好跟 构造函数 的调用顺序相反。

继承中的析构函数

```
class Animal {
public:
    Animal() {
        cout << "call Aniaml" << endl;
    }
    ~Animal() {
        cout << "call ~Aniaml" << endl;
    }
};

class Monkey :public Animal {
public:
    Monkey() {
        cout << "call Monkey" << endl;
    }
    ~Monkey() {
        cout << "call ~Monkey" << endl;
    }
};

int main() {
    Monkey m1;
    return 0;
}
```

```
call Aniaml
call Monkey
call ~Monkey
call ~Aniaml
```

多继承

在 C++ 中，当派生类只有一个基类时，称为单继承，同时，在 C++ 中，也是支持多继承的，多继承，也就是一个派生类有多个基类。

多继承容易让代码逻辑复杂、思路混乱，一直备受争议，中小型项目中较少使用，后来的某些高级语言干脆取消了多继承。

多继承

语法

```
class D: public A, private B, protected C{  
    //类D新增加的成员  
}
```

多继承 简单案例

```
class Animal{
```

```
};
```

```
class Person{
```

```
};
```

```
class Xtq:public Animal,public Person{
```

```
};
```

多继承 简单案例

```
class Animal{
public:
    Animal(){}

public:
    string nickName;
};
class Person{
public:
    Person(){}

public:
    string name;
    int age;
};
class Xtq:public Animal,public Person{
public:
    Xtq(){}
    void printInfo(){
        cout << "昵称=" << this->nickName << " 姓名=" << this->name << " 年龄=" << this->age << endl;
    }

private:
    string sex;
};
int main(){
    Xtq x1;
    x1.nickName = "xt";
    x1.name = "xtq";
    x1.age = 10000;
    x1.printInfo();
    return 0;
}
```

多继承 简单案例

子类Xtq,继承了所有的非私有的成员属性和方法。

多继承构造函数

在 C++ 中，多继承形式下的构造函数和单继承形式基本相同，只是要在派生类的构造函数中调用多个基类的构造函数。

基类构造函数的调用顺序和它们在派生类构造函数中出现的顺序无关，而是和声明派生类时基类出现的顺序相同。

语法

```
D(paramlist): A(paramlist1), B(paramlist2), C(paramlist3){  
    //  
}
```

多继承构造函数

```
class Person{
public:
    Person(string name):name(name){}

protected:
    string name;
};
class Worker{
public:
    Worker(double salary):salary(salary){}
protected:
    double salary;
};
class Ai:public Person,public Worker{
public:
    Ai(string name,double salary,string nickName): Person(name),Worker(salary),nickName(nickName){
    }
    void printInfo(){
        cout << this->nickName << endl;
        cout << this->name << endl;
        cout << this->salary << endl;
    }
private:
    string nickName;
};
int main(){
    Ai a1("zx",1000,"jqr");
    a1.printInfo();
    return 0;
}
```

调用Person和Worker的构造函数来初始化Ai的成员变量。

多继承构造函数

```
class Person{
public:
    Person(){
        cout << "call Person" << endl;
    }
};
class Worker{
public:
    Worker(){
        cout << "call Worker" << endl;
    }
};
class Ai:public Person,public Worker{
public:
    Ai(){
        cout << "call Ai" << endl;
    }
};
int main(){
    Ai a1;
    return 0;
}
```

```
call Person
call Worker
call Ai
```

多继承构造函数

```
class Person{
public:
    Person(){
        cout << "call Person" << endl;
    }
};
class Worker{
public:
    Worker(){
        cout << "call Worker" << endl;
    }
};
class Ai:public Person,public Worker{
public:
    Ai(): Person(),Worker(){
        cout << "call Ai" << endl;
    }
};
int main(){
    Ai a1;
    return 0;
}
```

效果和不写是一样的

```
call Person
call Worker
call Ai
```

多继承构造函数

```
class Person{
public:
    Person(){
        cout << "call Person" << endl;
    }
};
class Worker{
public:
    Worker(){
        cout << "call Worker" << endl;
    }
};
class Ai:public Person,public Worker{
public:
    Ai(): Worker(),Person(){
        cout << "call Ai" << endl;
    }
};
int main(){
    Ai a1;
    return 0;
}
```

调换父类构造函数顺序后，效果还是一样的

```
call Person
call Worker
call Ai
```

多继承构造函数

```
class Person{
public:
    Person(){
        cout << "call Person" << endl;
    }
};
class Worker{
public:
    Worker(){
        cout << "call Worker" << endl;
    }
};
class Ai:public Worker,public Person{
public:
    Ai(): Worker(),Person(){
        cout << "call Ai" << endl;
    }
};
int main(){
    Ai a1;
    return 0;
}
```

调换父类继承顺序后，效果不一样

```
call Worker
call Person
call Ai
```

多继承中，基类构造函数的调用顺序和它们在派生类构造函数中出现的顺序无关，而是和声明派生类时基类出现的顺序相同。

多继承命名冲突

在 C++ 中，多继承 如果多个基类中有相同的 成员变量 或者 成员函数，那么此时就会存在成员名字冲突的问题，如果存在名字冲突，我们在调用时，就必须显式声明需要调用的成员。

```
class Person{
public:
    Person(){}
    void printInfo(){}
};
class Worker{
public:
    Worker(){}
    void printInfo(){}
};
class Ai:public Worker,public Person{
public:
    Ai(): Worker(),Person(){}
};
int main(){
    Ai a1;
    a1.printInfo();
    return 0;
}
```

[错误] request for member 'printInfo' is ambiguous

[说明] candidates are: 'void Person::printInfo()'

[说明] 'void Worker::printInfo()'

```
class Person{
public:
    Person(){}
    void printInfo(){}
};
class Worker{
public:
    Worker(){}
    void printInfo(){}
};
class Ai:public Worker,public Person{
public:
    Ai(): Worker(),Person(){}
    void func(){
        Person::printInfo();
        Worker::printInfo();
    }
};
int main(){
    Ai a1;
    a1.func();
    return 0;
}
```

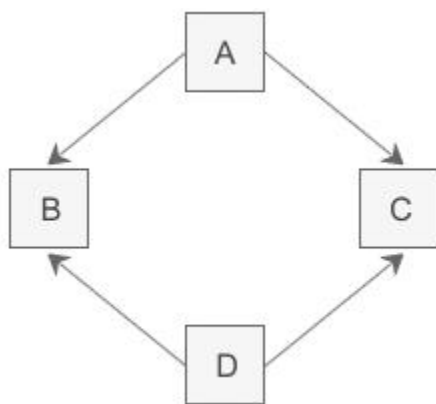


正常运行

菱形继承

在 C++ 中，在使用多继承时，如果发生了如果类 A 派生出类 B 和类 C，类 D 继承自类 B 和类 C，这时候就发生了菱形继承。

如果发生了菱形继承，这个时候类 A 中的成员变量和成员函数继承到类 D 中变成了两份，一份来自 A→B→D 这条路径，另一份来自 A→C→D 这条路径。



菱形继承

```
// 间接基类A
class A
{
protected:
    int m_a;
};

// 直接基类B
class B: public A
{
protected:
    int m_b;
};

// 直接基类C
class C: public A
{
protected:
    int m_c;
};
```

```
// 派生类D
class D: public B, public C
{
public:
    void setb(int b)
    {
        m_b = b; //正确
    }
    void setc(int c)
    {
        m_c = c; //正确
    }
    void setd(int d)
    {
        m_d = d; //正确
    }

private:
    int m_d;
};

int main()
{
    D d;
    return 0;
}
```

菱形继承

//派生类D

```
class D: public B, public C
```

```
{
```

```
public:
```

```
    void seta(int a)
```

```
    {
```

```
        m_a = a; //命名冲突
```

```
    }
```

```
    void setb(int b)
```

```
    {
```

```
        m_b = b; //正确
```

```
    }
```

```
    void setc(int c)
```

```
    {
```

```
        m_c = c; //正确
```

```
    }
```

```
    void setd(int d)
```

```
    {
```

```
        m_d = d; //正确
```

```
    }
```

```
private:
```

```
    int m_d;
```

```
};
```

菱形继承

//派生类D

```
class D: public B, public C
```

```
{
```

```
public:
```

```
    void seta(int a)
```

```
    {
```

```
        A::m_a = a; //命名不在冲突
```

```
    }
```

```
    void setb(int b)
```

```
    {
```

```
        m_b = b; //正确
```

```
    }
```

```
    void setc(int c)
```

```
    {
```

```
        m_c = c; //正确
```

```
    }
```

```
    void setd(int d)
```

```
    {
```

```
        m_d = d; //正确
```

```
    }
```

```
private:
```

```
    int m_d;
```

```
};
```

虚继承

在 C++ 中，在使用多继承时，如果发生了菱形继承，那么就会出现数据冗余的问题，为了解决菱形继承出现的数据冗余的问题，C++ 提出了虚继承，虚继承使得派生类中只保留一份间接基类的成员。

语法

```
class B: virtual public A{ //虚继承  
};
```

虚继承

// 间接基类A

```
class A
{
protected:
    int m_a;
};
```

// 直接基类B

class B: virtual public A

```
{
protected:
    int m_b;
};
```

// 直接基类C

class C: virtual public A

```
{
protected:
    int m_c;
};
```

//派生类D

class D: public B, public C

{
public:

void seta(int a)

{

m_a = a; //命名冲突?

}

void setb(int b)

{

m_b = b; //正确

}

void setc(int c)

{

m_c = c; //正确

}

void setd(int d)

{

m_d = d; //正确

}

private:

int m_d;

};

虚继承构造函数

在 C++ 中，普通的继承时，可以在子类直接显式的调用父类的构造函数，在虚继承中，虚基类是由最终的派生类初始化的。

也就是说，最终派生类的构造函数必须要调用虚基类的构造函数。对最终的派生类来说，虚基类是间接基类，而不是直接基类。这跟普通继承不同，在普通继承中，派生类构造函数中只能调用直接基类的构造函数，不能调用间接基类的。

虚继承构造函数

```
//虚基类A
class A
{
public:
    A(int a):m_a(a)
    {
    }

protected:
    int m_a;
};

//直接派生类B
class B: virtual public A
{
public:
    B(int a, int b): A(a), m_b(b)
    {
    }

public:
    void display()
    {
        cout<<"Call B m_a = " << m_a << ", m_b = " << m_b << endl;
    }

protected:
    int m_b;
};

//直接派生类C
class C: virtual public A
{
public:
    C(int a, int c):A(a), m_c(c)
    {
    }

public:
    void display()
    {
        cout << "Call C m_a = " << m_a << ", m_c = " << m_c << endl;
    }

protected:
    int m_c;
};
```

虚继承构造函数

```
//间接派生类D
class D: public B, public C
{
public:
```

```
    D(int a, int b, int c, int d): A(a), B(90, b), C(100, c), m_d(d)
    {
    }
```

```
public:
```

```
    void display()
    {
```

```
        cout<<"Call D m_a = " << m_a << ", m_b = " << m_b << ", m_c = " << m_c << ", m_d
```

```
    = " << m_d << endl;
    }
```

```
private:
```

```
    int m_d;
```

```
};
```

```
int main()
```

```
{
```

```
    B b(10, 20);
```

```
    b.display();
```

```
    C c(30, 40);
```

```
    c.display();
```

```
    D d(50, 60, 70, 80);
```

```
    d.display();
```

```
    return 0;
```

```
}
```

m_a 最终被初始化为 50，这正是在 D 中直接调用 A 的构造函数的结果

```
Call B m_a = 10, m_b = 20
Call C m_a = 30, m_c = 40
Call D m_a = 50, m_b = 60, m_c = 70, m_d = 80
```

练习题 题目

定义一个基类Animal，它包含两个私有数据成员，一个是string，存储动物的名称（例如"Fido"或"Yogi"），另一个是整数成员weight，包含该动物的重量（单位是磅）。该类还包含一个公共成员函数who()，它可以显示一个消息，给出Animal对象的名称和重量。把Animal用作公共基类，派生两个类Lion和Aardvark。再编写一个main()函数，创建Lion和Aardvark对象（"Leo"，400磅；"Algernon"，50磅）。为派生类对象调用who()成员，说明who()成员在两个派生类中是继承得来的。

练习题 代码

```
#include <iostream>
using namespace std;
class Animal{
public:
    Animal(string name,int wt):name(name),weight(wt){}
    void who(){
        cout << "name is " << name << " weight is " << weight << endl;
    }
private:
    string name;
    int weight;
};
class Lion: public Animal{
public:
    Lion(string name,int wt): Animal(name,wt){}
};
class Aardvark: public Animal{
public:
    Aardvark(string name,int wt): Animal(name,wt){}
};
int main()
{
    Lion lion1("lion",500);
    Aardvark aardvark1("aar1",100);
    lion1.who();
    aardvark1.who();

    return 0;
}
```

知识点总结

- | | |
|-------------------|----|
| 1. 继承的基本概念和派生类的定义 | 了解 |
| 2. 继承的语法规则 | 熟悉 |
| 3. 三种继承方式 | 重点 |
| 4. 继承中的构造和析构 | 重点 |
| 5. 多继承 | 重点 |
| 6. 多继承构造函数 | 重点 |
| 7. 多继承命名冲突 | 重点 |
| 8. 菱形继承 | 重点 |
| 9. 虚继承 | 重点 |
| 10. 虚继承构造函数 | 重点 |

作业

1. 派生类从基类那里继承了什么？
2. 派生类不能从基类那里继承什么？

作业

1. 派生类从基类那里继承了什么？

基类的公有成员将成为派生类的公有成员，基类的私有对象也将成为派生类的一部分，但只能通过基类的公有和保护方法访问。

2. 派生类不能从基类那里继承什么？

- 构造函数
- 赋值运算符
- 析构函数
- 友元

下节课预告

类的多态